NAME : Maaz Aslam Chogle

ROLL NO : 21CO19

BATCH NO : 03

Date of performance : September 29, 2025

EXPERIMENT NO : 03

AIM : Design a navbar using CSS and add JavaScript to highlight active links and implement a dropdown menu

Theory:

A navigation menu (or navbar) is a fundamental component of any website, allowing users to move between different sections or pages. Making it interactive enhances the user experience.

1. **HTML (HyperText Markup Language):** This provides the **structure** of the navbar. We use semantic tags like `<nav>` to define the navigation block, `<ul>` for an unordered list of menu items, `<li>` for each individual list item, and `<a>` (anchor) for the actual clickable links. A dropdown menu is simply a nested `<ul>` inside an `<li>`.
2. **CSS (Cascading Style Sheets):** This is responsible for the **presentation and styling**. We use CSS to control the layout, colors, fonts, and spacing of the navbar. Key concepts include:
   - **Flexbox (`display: flex`)**: To easily align navigation items horizontally.
   - **Pseudo-classes (`:hover`)**: To change the style of a link when a user hovers their mouse over it.
   - **Positioning (`position: relative`, `position: absolute`)**: Crucial for placing the dropdown menu directly below its parent item.
   - **Visibility (`display: none`, `display: block`)**: To hide the dropdown menu by default and show it when needed.
3. **JavaScript (JS):** This adds **behavior and interactivity**. While CSS can handle simple hover effects, JavaScript is needed for more complex logic.
   - **Active Link Highlighting**: JS checks the current page's URL and compares it with the `href` attribute of each link in the navbar. If a match is found, it adds a special CSS class (e.g., `.active`) to that link to visually highlight it.

- **Dropdown Menu Control**: We use an event listener (like `click`) on the dropdown's parent link. When the user clicks it, a JavaScript function toggles a CSS class (e.g., `.show`) on the dropdown menu, which makes it visible or hidden. This is more accessible and mobile-friendly than a pure CSS `hover` implementation.

Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Navbar with JavaScript Dropdown</title>

    <style>

        * {

            margin: 0;

            padding: 0;

            box-sizing: border-box;

            font-family: Arial, sans-serif;

        }



        html, body {

            margin: 0;

            padding: 0;

        }
```

```
body {
    background-color: #f5f5f5;
}

.navbar {
    background-color: #2c3e50;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 20px;
}

.nav-list {
    list-style: none;
    display: flex;
}

.nav-item {
    position: relative;
    margin-right: 10px;
}

.nav-link {
```

```
    color: white;
    text-decoration: none;
    padding: 10px 15px;
    display: block;
    border-radius: 4px;
    transition: background-color 0.3s;
}

.nav-link:hover {
    background-color: #34495e;
}

.nav-link.active {
    background-color: #3498db;
}

/* Dropdown styles */
.dropdown-content {
    position: absolute;
    background-color: white;
    min-width: 160px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2);
    border-radius: 4px;
```

```
    display: none;
    top: 100%;
    left: 0;
    z-index: 1;
}

.dropdown-content a {
    color: #333;
    padding: 10px 15px;
    text-decoration: none;
    display: block;
    border-bottom: 1px solid #eee;
}

.dropdown-content a:hover {
    background-color: #f0f0f0;
}

.dropdown-content.show {
    display: block;
}

.content {
```

```
            padding: 20px;
            background-color: white;
            border-radius: 8px;
            box-shadow: 0 2px 5px rgba(0,0,0,0.1);
        }

        h1 {
            color: #2c3e50;
            margin-bottom: 15px;
        }

        p {
            line-height: 1.6;
            color: #555;
            margin-bottom: 10px;
        }
    </style>
</head>
<body>
    <!-- Navbar -->
    <nav class="navbar">
        <ul class="nav-list">
            <li class="nav-item">
```

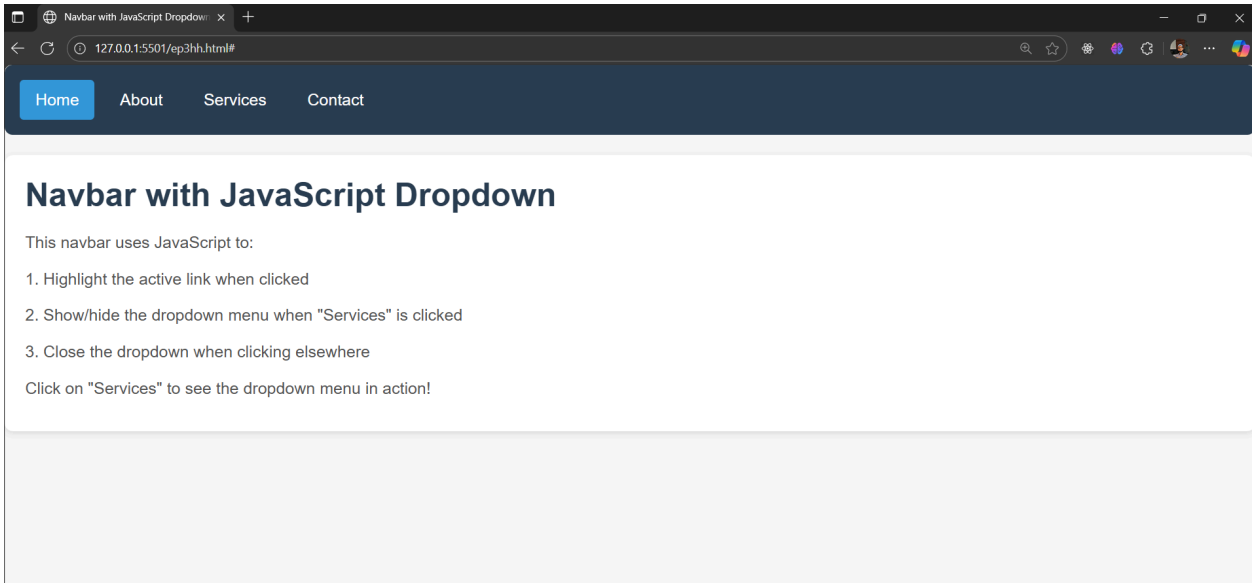
```
            <a href="#" class="nav-link active">Home</a>
        </li>
        <li class="nav-item">
            <a href="#" class="nav-link">About</a>
        </li>
        <li class="nav-item">
            <a href="#" class="nav-link">Services</a>
            <div class="dropdown-content">
                <a href="#">Web Design</a>
                <a href="#">Development</a>
                <a href="#">SEO</a>
            </div>
        </li>
        <li class="nav-item">
            <a href="#" class="nav-link">Contact</a>
        </li>
    </ul>
</nav>

<div class="content">
    <h1>Navbar with JavaScript Dropdown</h1>
    <p>This navbar uses JavaScript to:</p>
    <p>1. Highlight the active link when clicked</p>
```

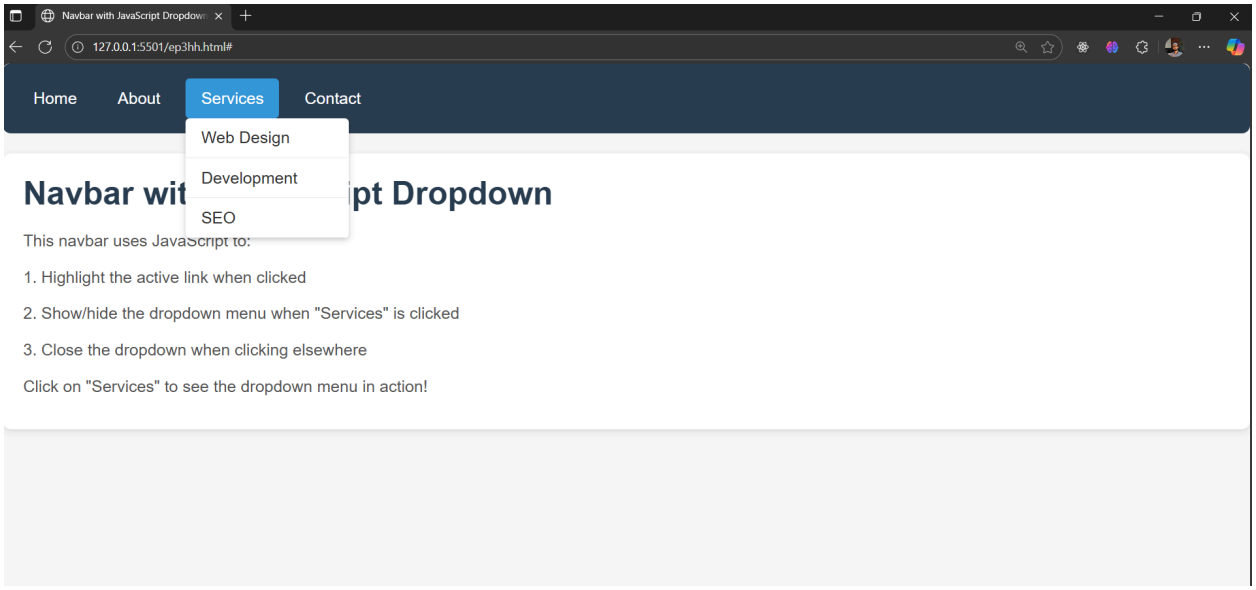
```
        <p>2. Show/hide the dropdown menu when "Services" is clicked</p>

        <p>3. Close the dropdown when clicking elsewhere</p>

        <p>Click on "Services" to see the dropdown menu in action!</p>

    </div>



    <script>

        // Get all navigation links

        const navLinks = document.querySelectorAll('.nav-link');

        // Get the dropdown content

        const dropdownContent =
document.querySelector('.dropdown-content');

        // Get the Services link (the one with the dropdown)

        const servicesLink =
document.querySelector('.nav-item:nth-child(3) .nav-link');



        // Function to set active link

        function setActiveLink() {

            // Remove active class from all links

            navLinks.forEach(link => {

                link.classList.remove('active');

            });



            // Add active class to clicked link

            this.classList.add('active');
```

```
}

// Function to toggle dropdown
function toggleDropdown() {
    dropdownContent.classList.toggle('show');
}

// Function to close dropdown when clicking outside
function closeDropdown(e) {
    if (!e.target.closest('.nav-item:nth-child(3)')) {
        dropdownContent.classList.remove('show');
    }
}

// Add event listeners
navLinks.forEach(link => {
    link.addEventListener('click', setActiveLink);
});

servicesLink.addEventListener('click', function(e) {
    e.preventDefault(); // Prevent link from navigating
    toggleDropdown();
});
```

```
        // Close dropdown when clicking elsewhere on the page
        document.addEventListener('click', closeDropdown);
    </script>
</body>
</html>
```

Output:





Conclusion:

We have successfully designed and implemented a modern, interactive navigation bar.

- **HTML** was used to create the fundamental structure of the menu, including a nested list for the dropdown.
- **CSS** was applied to style the navbar, creating a clean visual appearance, hover effects, and the necessary classes (`.active`, `.show`) for dynamic changes.
- **JavaScript** provided the core interactivity by dynamically adding the `.active` class to the current page's link and managing the visibility of the dropdown menu on a user's click.

This experiment effectively demonstrates how these three core web technologies work together to build responsive and user-friendly web components, enhancing the overall navigation experience for the user.